Building a Site-to-Site VPN in AWS

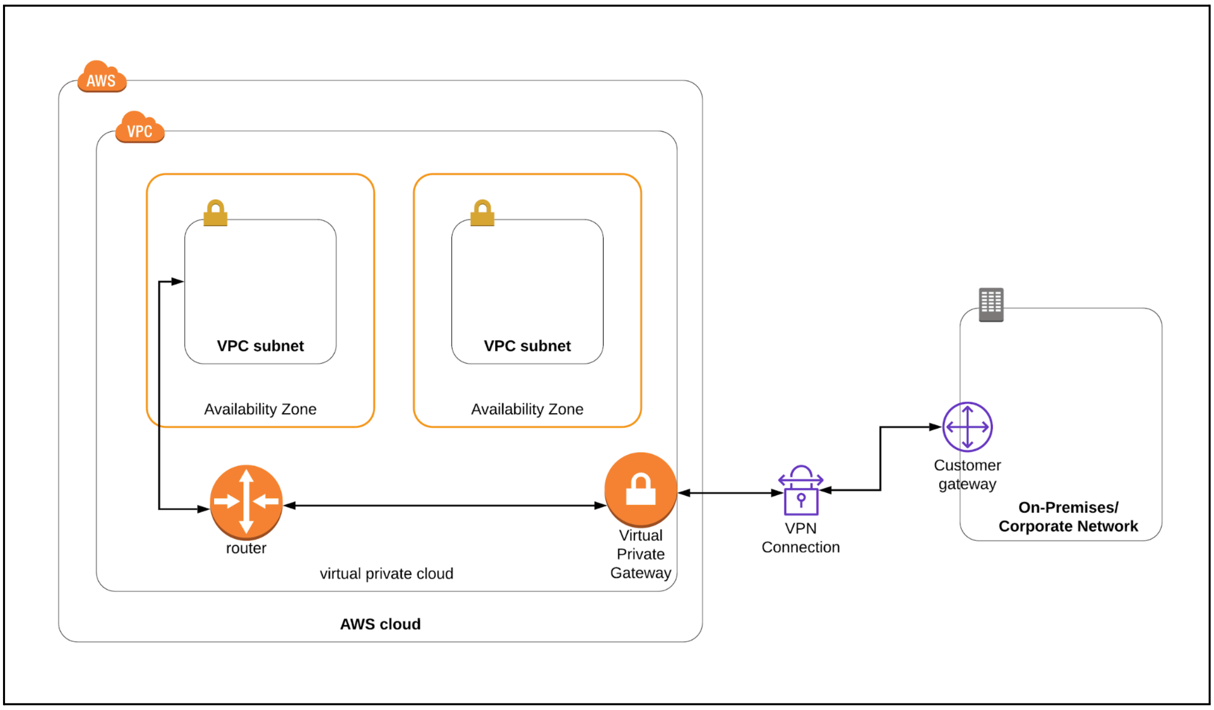
🌐 Introduction

In real-world enterprise environments, companies often need to connect their **on-premises data centers** to their **AWS Cloud VPCs** securely.
AWS Site-to-Site VPN provides a reliable and encrypted tunnel using the IPsec protocol, enabling private communication between these two networks.

But what if you don't have a physical on-premises network? In this lab, we will **simulate an on-premises environment using a second VPC in the same AWS region (us-east-1)**.
One VPC will represent the **corporate/on-premises side**, while the other will act as the **AWS Cloud network**. We will then connect them using a Site-to-Site VPN.

What is AWS Site-to-Site VPN?

A VPN connection refers to the connection between your VPC and your own on-premises network. Site-to-Site VPN supports Internet Protocol security (IPsec) VPN connections.
By default, instances that you launch into an Amazon VPC can't communicate with your own (remote/on-premises) network.



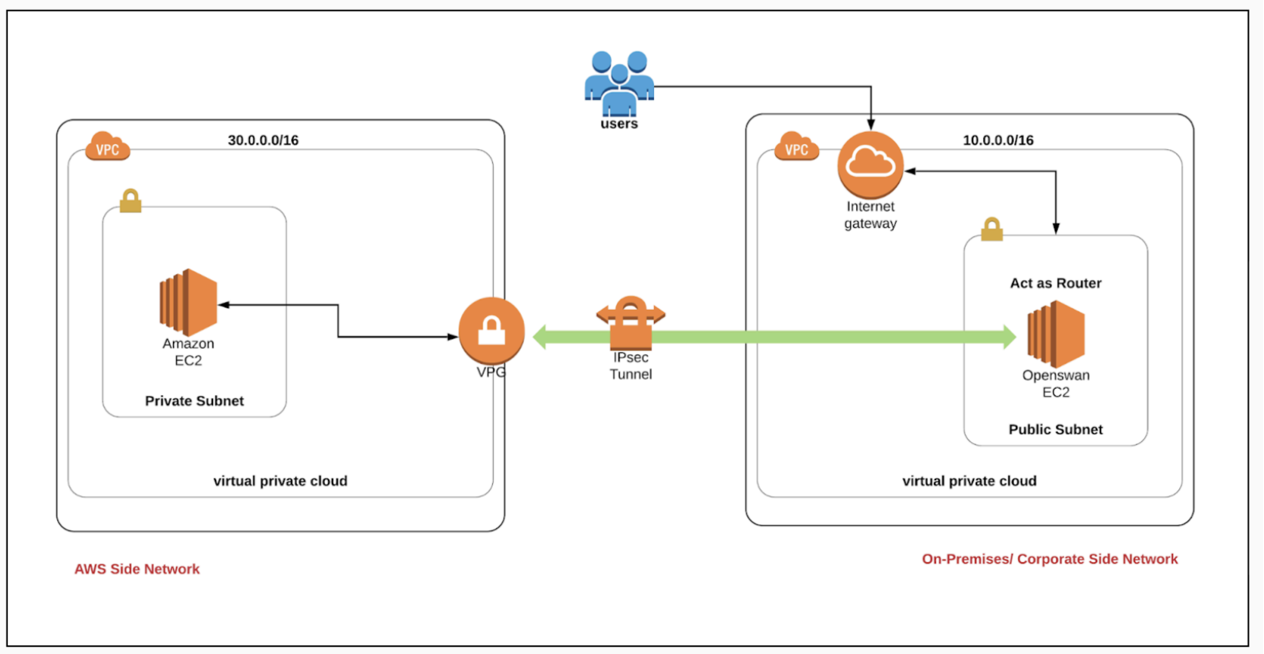
- **VPN connection:** A secure connection between your on-premises equipment and your VPCs.
- **VPN tunnel:** An encrypted link where data can pass from the customer network to or from AWS.
- **Customer gateway:** An AWS resource which provides information to AWS about your customer gateway device.
- **Customer gateway device:** A physical device or software application on your side of the Site-to-Site VPN connection.
- **Virtual private gateway:** The VPN concentrator on the Amazon side of the Site-to-Site VPN connection. You use a virtual private gateway or a transit gateway as the gateway for the Amazon side of the Site-to-Site VPN connection.

🏗️ Architecture Overview

Our lab consists of two VPCs in **N. Virginia (us-east-1)**:

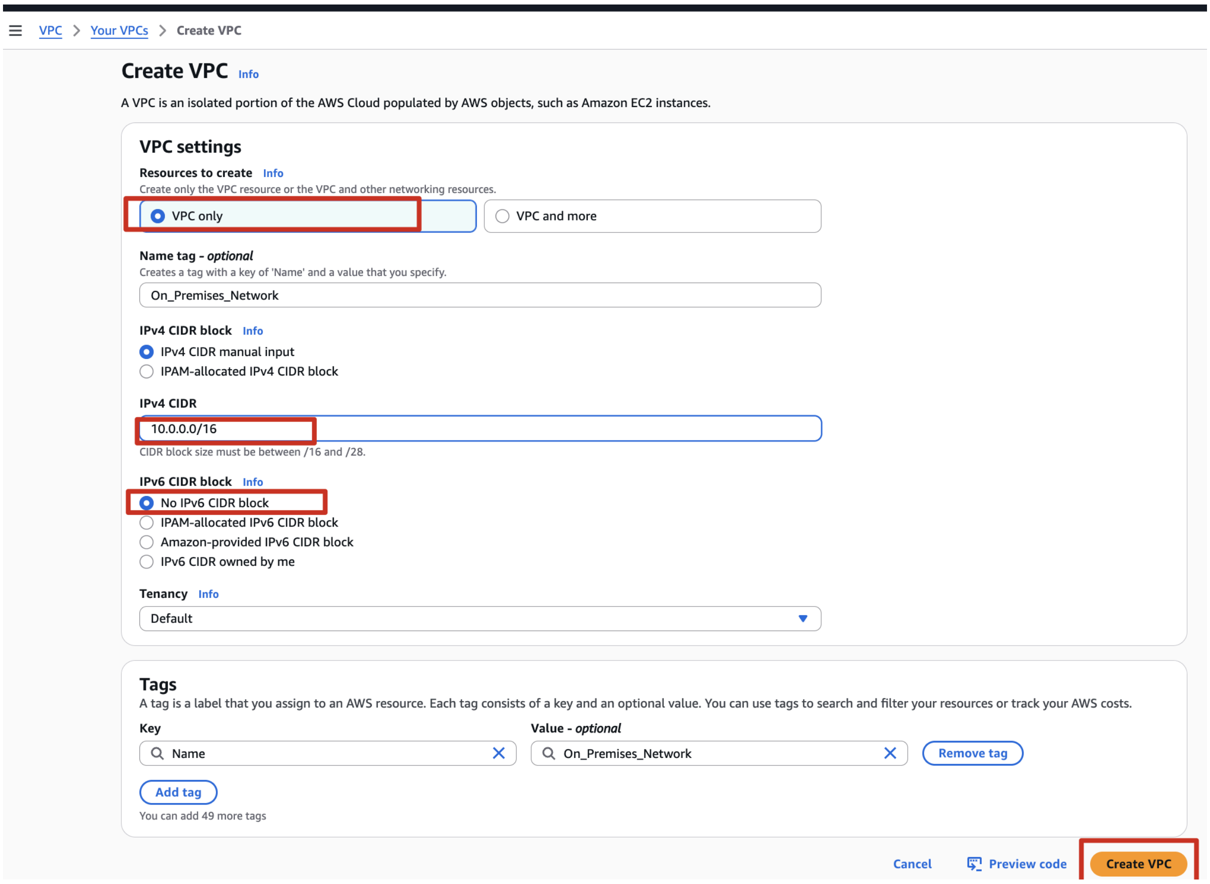
- **VPC-OnPrem (Corporate Simulation)**
  - Public subnet
  - EC2 instance with **Openswan** (acting as a customer router)
- **VPC-AWS (AWS Cloud Network)**
  - Private subnet
  - EC2 instance to test private connectivity
- **VPN Connection**:
  - **Customer Gateway (CGW)** on AWS side (logical representation of the on-prem router)
  - **Virtual Private Gateway (VGW)** on AWS side (VPN concentrator)
  - **Two VPN tunnels** (AWS always provides redundancy, though in this lab we will focus on one active tunnel)

📌 *Tip: Always make sure your VPC CIDR blocks do not overlap; otherwise, routing will fail.*



Step 1: Create the On-Premises Simulation VPC.

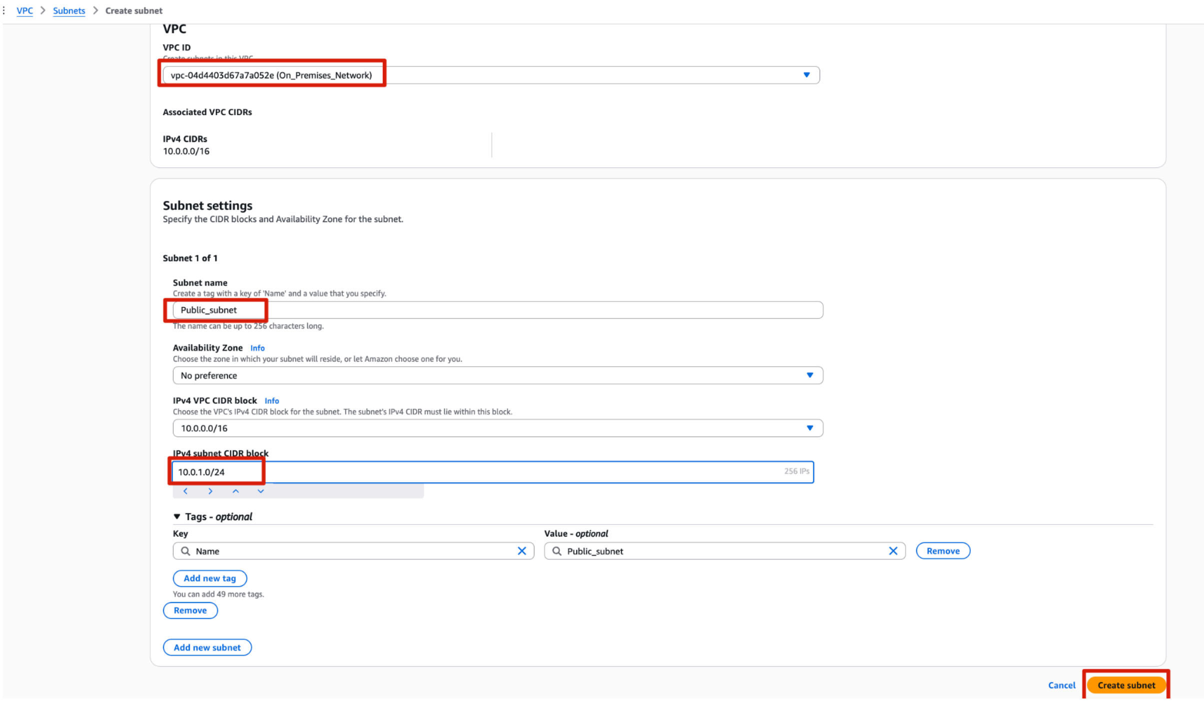
1. Go to **VPC > Create VPC**
2. Navigate to **Your VPC's** on the left panel and click on the **Create VPC**.
    - Select **VPC only**
    - Name tag : Enter ***On_Premises_Network***
    - IPv4 CIDR block : Enter ***10.0.0.0/16***



3. Leave everything else as default and click on the **Create VPC**.

Step 2: Create a Public subnet

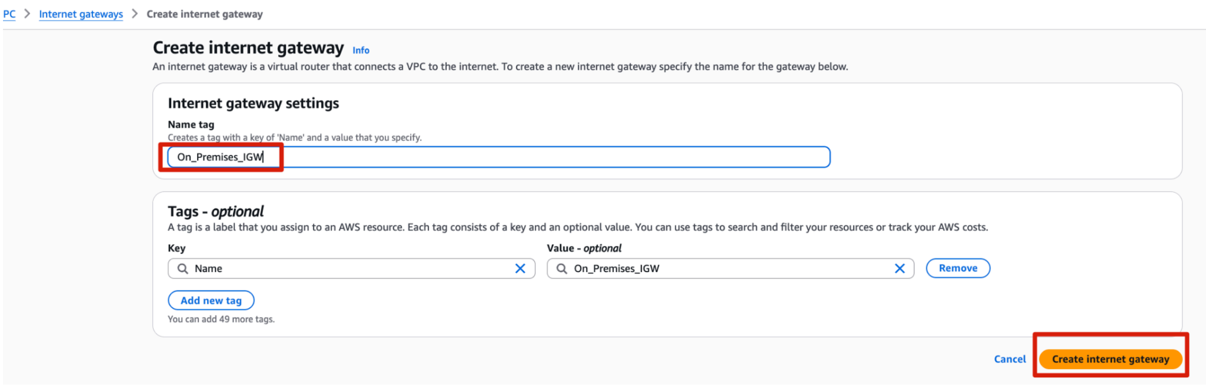
1. **Subnets → Create Subnet**.
   - VPC ID : Select the **On_Premises_Network** VPC from the list.
   - Subnet name : Enter ***Public_subnet***
   - Availability Zone :No preference
   - IPv4 CIDR block : Enter ***10.0.1.0/24***



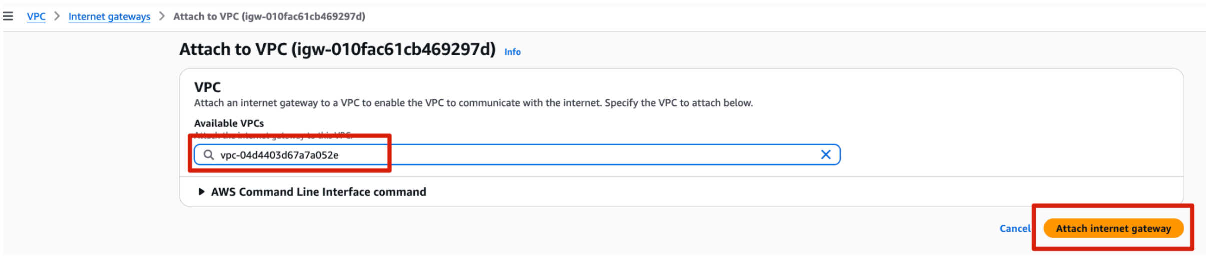
2. Now click on the **Create Subnet**

Step 3: Create and attach an Internet Gateway

1. **Internet Gateways → Create Internet Gateway**
   - Name tag : Enter ***On_Premises_IGW***
2. Click on the **Create Internet Gateway**

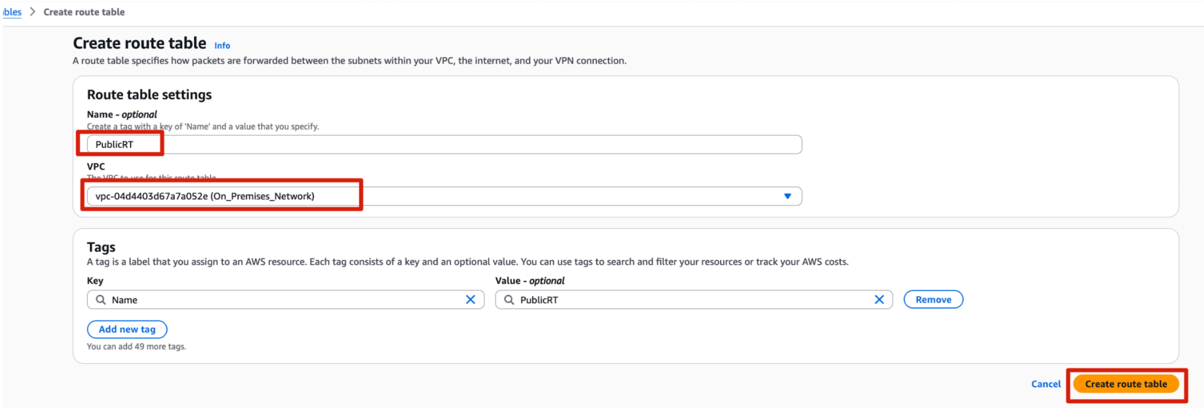


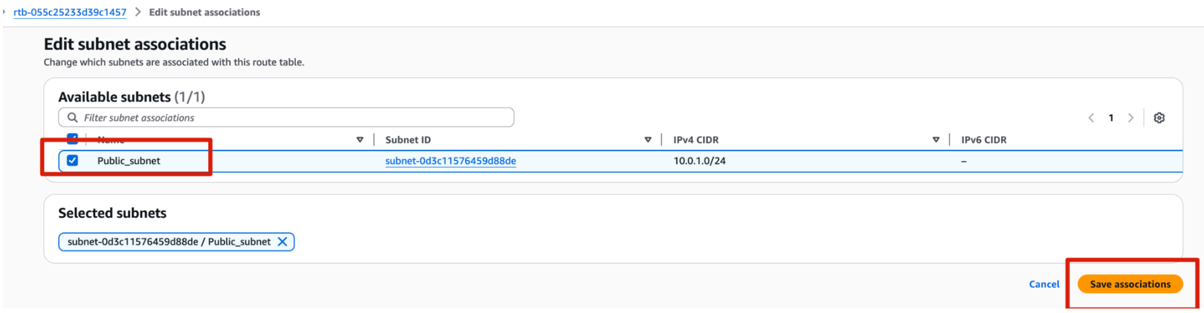
3. Now click on the **Actions** and select **Attach to VPC**.
    - Available VPCs : select **On_Premises_Network** from the list.
4. Click on the **Attach Internet Gateway**



Step 4: Create a Public Route Table and associate it with the subnet

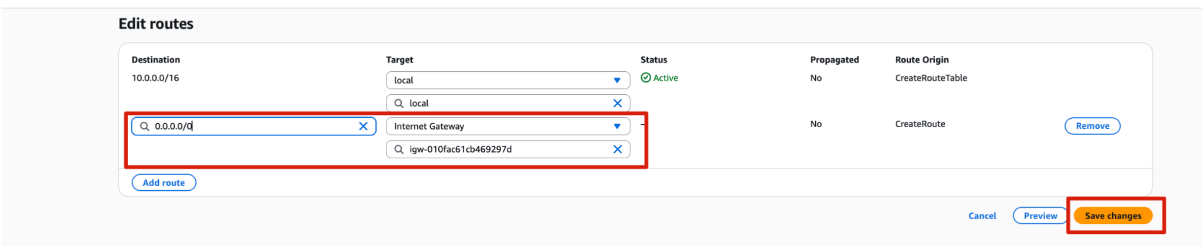
1. **Route Tables →Create route table** button.
    - Name tag : Enter ***PublicRT***
    - VPC* : Select the **On_Premises_Network** from the list.
2. Click on the **Create route table** button.
3. Now click on **Route Tables** in the left panel and select the **PublicRT** from the list and go to the **Subnet associations** tab in below.
4. Click on the **Edit subnet associations** button.
5. Now select the subnet **Public_subnet** and click on the **Save associations** button.





Step 5: Add public Route in the Route table

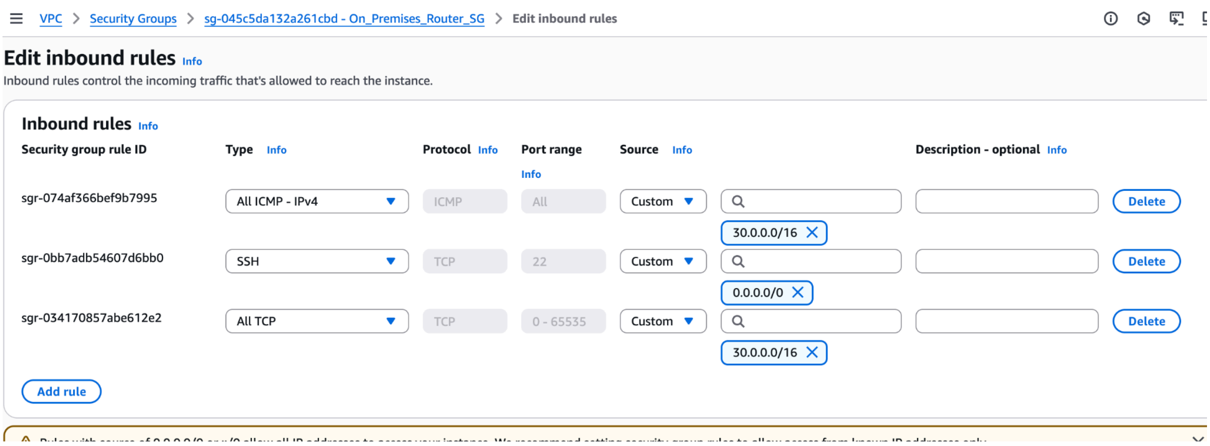
1. **Route Tables → PublicRT**
2. **Routes → Edit routes** button.
3. Now click on the **Add route** button.
   - Destination : Enter ***0.0.0.0/0***
   - Target : select **Internet Gateway** and then select the **Internet Gateway id.**



4. Click on the **Save changes** button.

Step 6: Create Security Group

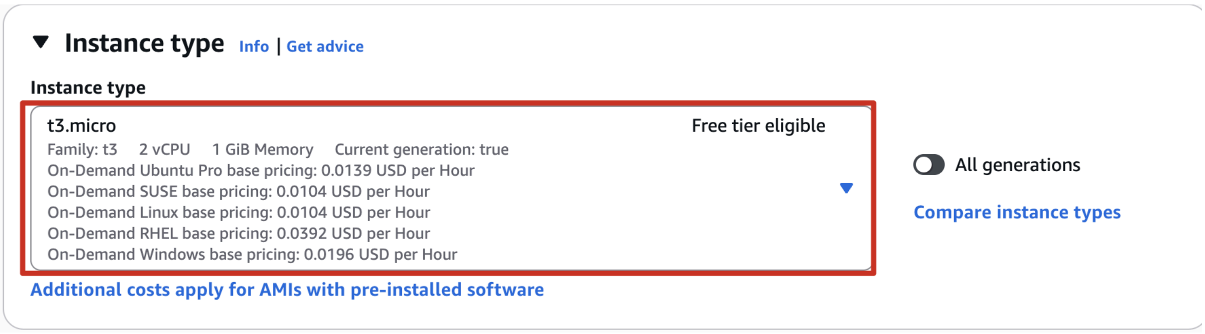
1. **EC2 → security groups → Create security group**
    - Security group name : Enter ***On_Premises_Router_SG***
    - Description : Enter ***Security group for public Router***
    - To add **SSH**,
        - Choose Type: **SSH**
        - Source: **Anywhere** (From ALL IP addresses accessible).
    - For **All TCP,**
        - Click on **Add Security group rule** button.
        - Choose Type: **All TCP**
        - Source: **Custom** and Enter ***30.0.0.0/16*** in the textbox.
    - For **IPv4 ICMP,**
        - Click on **Add Security group rule** button.
        - Choose Type: **All ICMP - IPv4**
        - Source: **Custom** and Enter ***30.0.0.0/16*** in the textbox.



Step 7: Launch an EC2 instance

1. **EC2→ Instances→ Launch Instances**
2. Enter Name as ***On_Premises_Router***
3. **Choose an Amazon Machine Image (AMI):** Search for **Amazon Linux 2023 kernal-6.1 AMI** in the search box and click on the **select** button.

4. Choose an Instance Type: select **t3.micro**

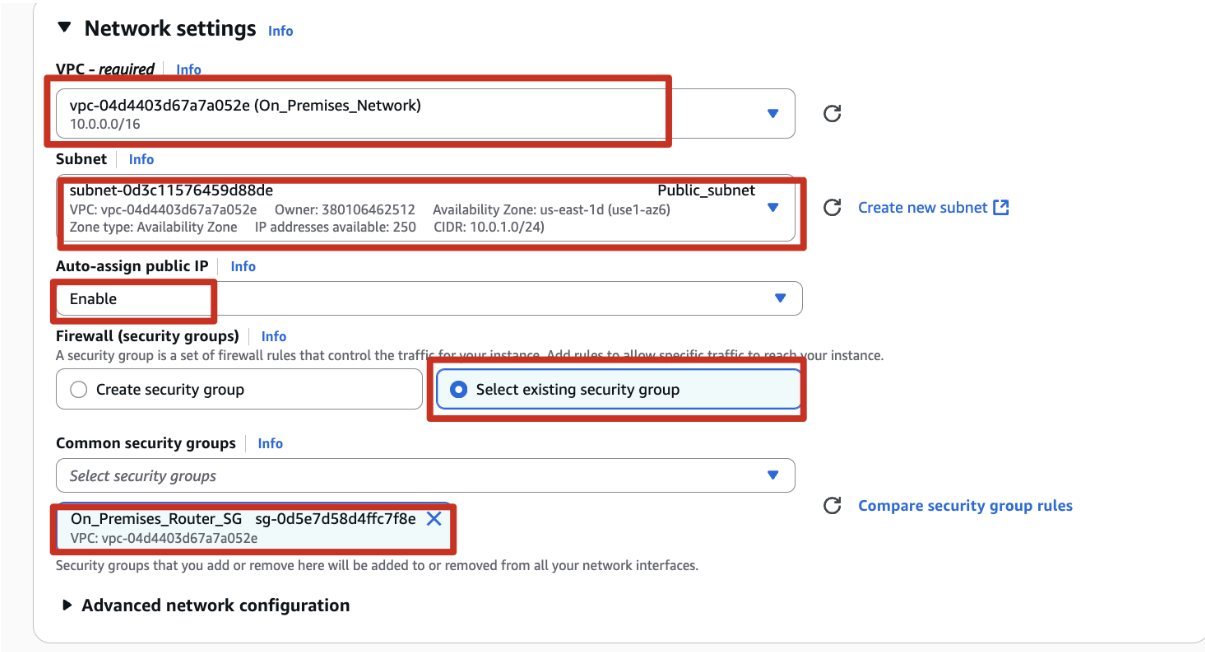


Key Pair : Choose **Create a new key Pair** from the dropdown list.

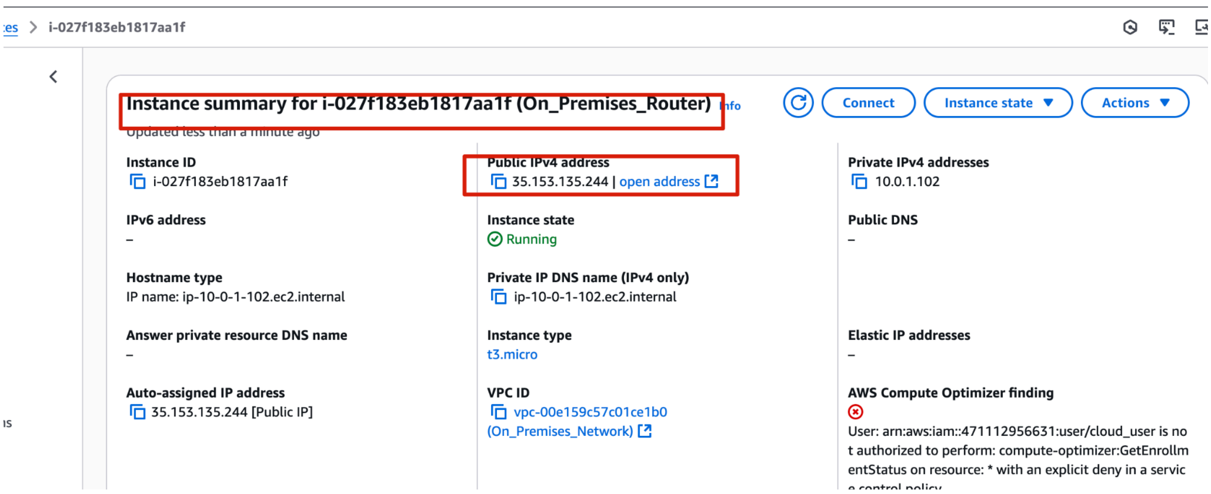
- Key pair name : Enter ***Router-key***
- Key Pair Type: Select **RSA**
- Click on **Create key pair** button to download the key to your local machine.

5. Under **Network Settings,** click on **Edit** button.

- Network : Select **On_Premises_Network**
- Subnet : leave as default
- Auto-assign Public IP : Select **Enable**



6. Configure Security Group:
    - Firewall(security groups) : Select **existing security group**
    - Select ***On_Premises_Router_SG***
7. Click on **Launch Instance** button.

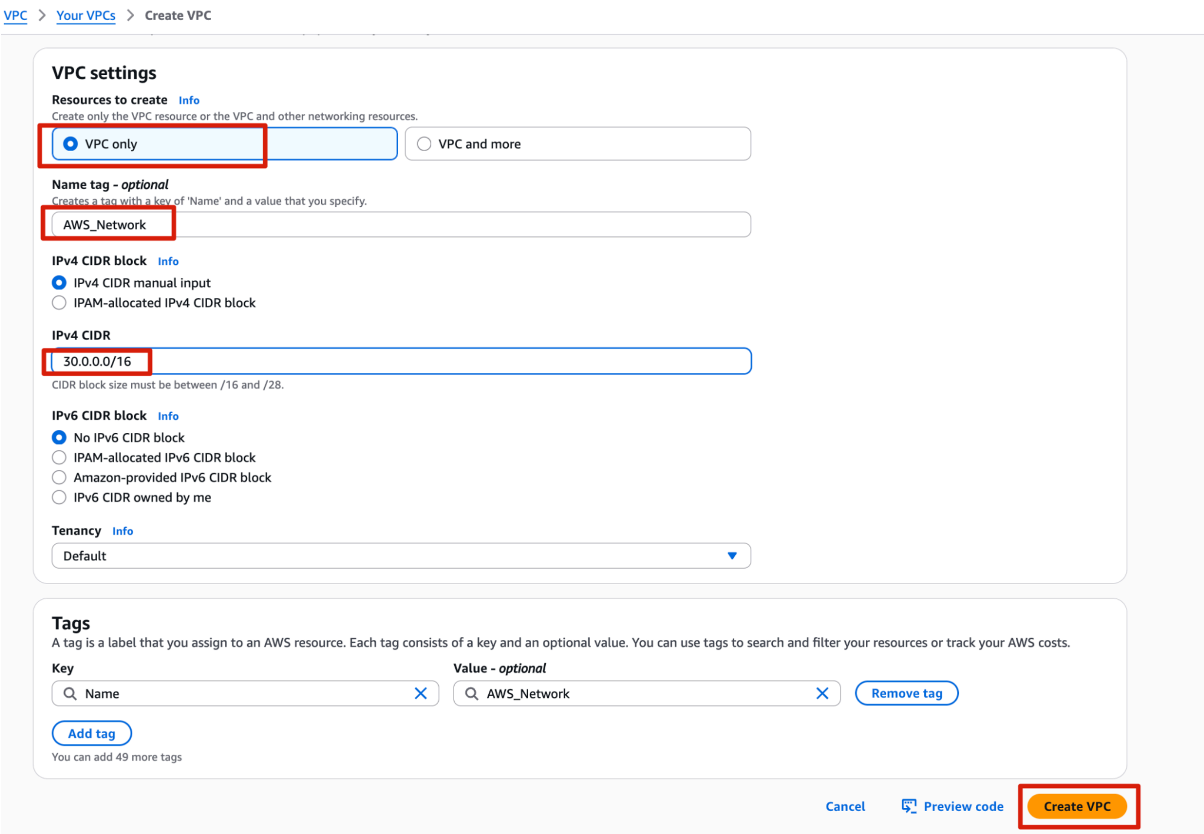


Step 8: Create AWS_Network VPC

1. **VPC →Your VPC → Create VPC**
    - Select **VPC Only**

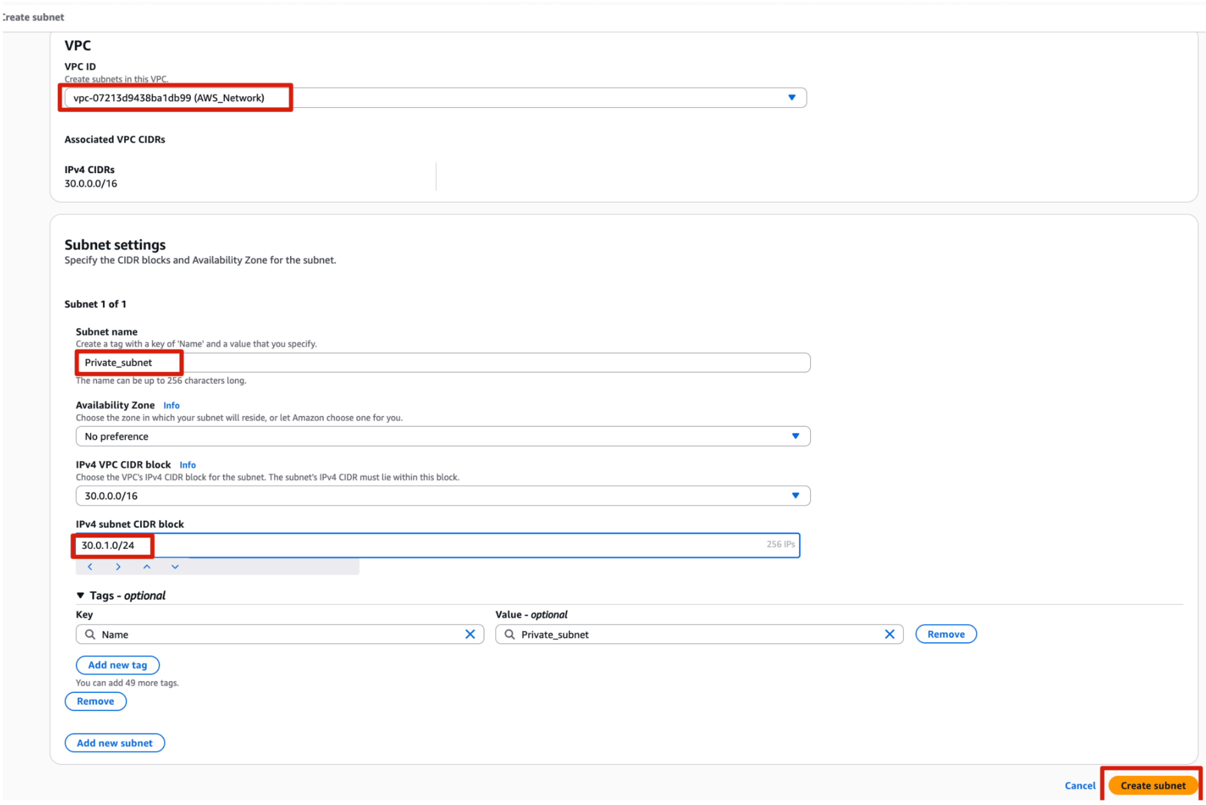
- Name tag : Enter ***AWS_Network***
- IPv4 CIDR block : Enter ***30.0.0.0/16***

2. Leave everything else as default and click on the **Create VPC**



Step 9: Create a Private subnet, a Private Route Table and associate it with the subnet

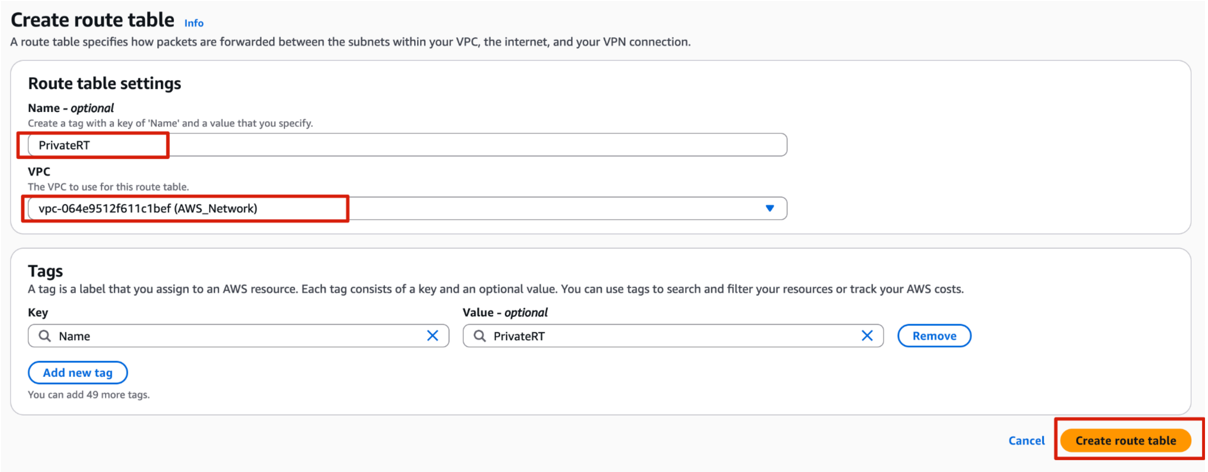
1. **Subnets →Create Subnets**
   - VPC ID : Select the **AWS_Network** VPC from the list.
   - Subnet name : Enter ***Private_subnet***
   - Availability Zone : No preference
   - IPv4 CIDR block : Enter ***30.0.1.0/24***



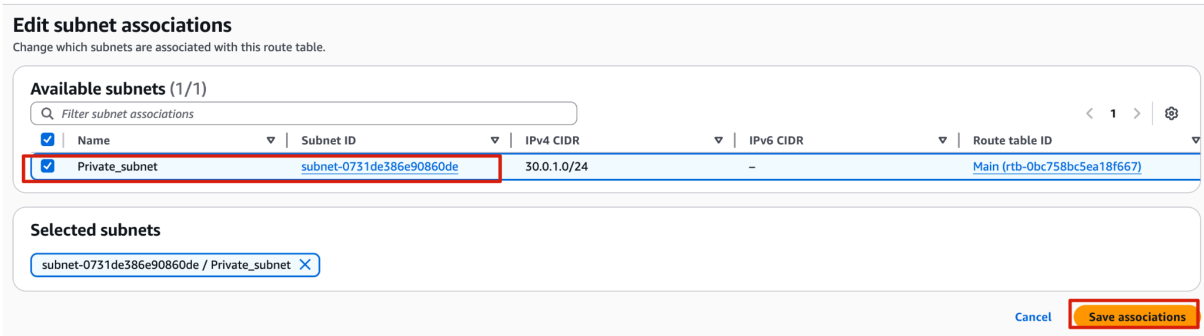
2. Now click on the **Create Subnet**

Create a Private Route Table and associate it with the subnet

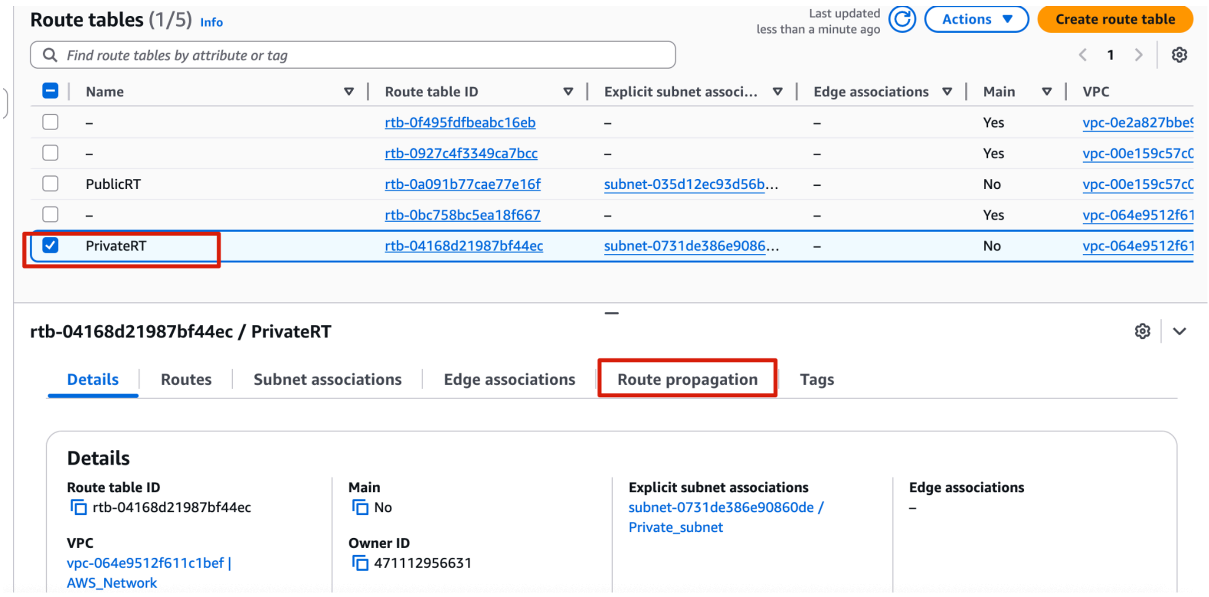
1. **Route Tables →Create route table**
    - Name tag : Enter ***PrivateRT***
    - VPC* : Select the **AWS_Network** from the list.



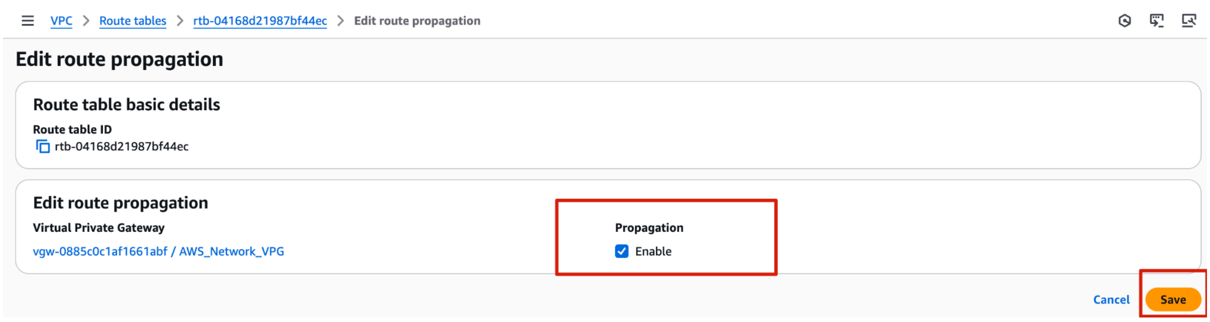
2. **Create route table→ Route Tables→ PrivateRT** from the list and go to the **Subnet associations** tab in below.
3. Click on the **Edit subnet associations** button.
4. Now select the subnet **Private_subnet** and click on the **Save associations** button.



Now select the PrivateRT **RouteTable**, where VPC is mentioned as **AWS_Network**. And goto the **Route propagation** tab below and click on the **Edit route propagation** button.



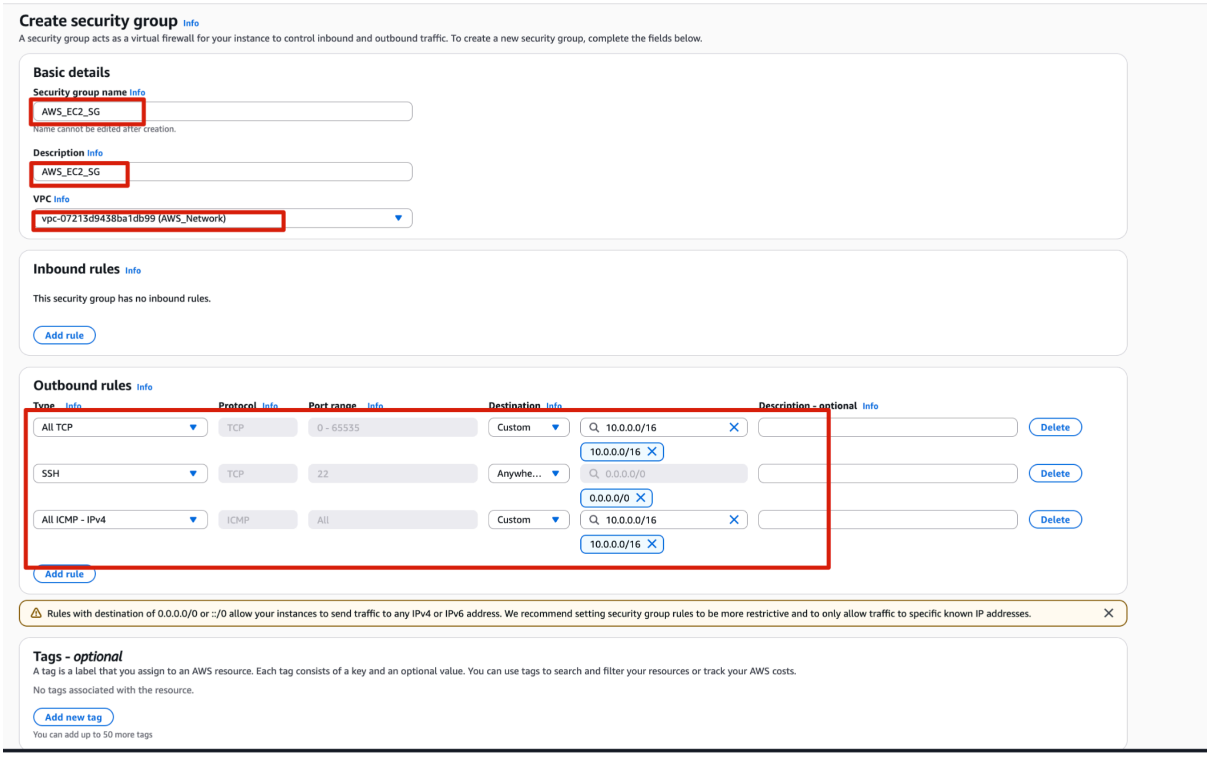
Check the **Enable** checkbox and click on the **Save** button.



Step 10: Create security group

1. **EC2 →security groups→ create security groups**.
    - Security group name : Enter ***AWS_EC2_SG***
    - Description : Enter ***Security group for public Router***
        - To add **SSH**,
            - Choose Type: **SSH**
            - Source: **Anywhere** (From ALL IP addresses accessible).
        - For **All TCP,**
            - Click on **Add Security group rule** button.
            - Choose Type: **All TCP**
            - Source: **Custom** and Enter

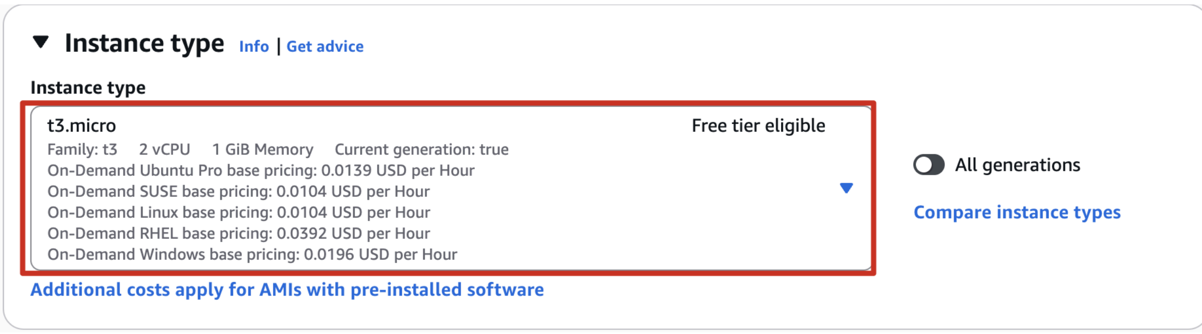
        - ***10.0.0.0/16***
        - in the textbox.
    - For **IPv4 ICMP,**
        - Click on **Add Security group rule** button.
        - Choose Type: **All ICMP - IPv4**
        - Source: **Custom** and Enter
        - ***10.0.0.0/16***
        - in the textbox.



Step 11: Launch an EC2 instance

1. **EC2 →Instances → Launch Instances**
2. Enter Name as ***AWS_EC2***
3. **Choose an Amazon Machine Image (AMI):** Search for **Amazon Linux 2023 kernel-6.1 AMI** in the search box and click on the **select** button.

4. **Choose an Instance Type:** select **t3.micro**



Key Pair : Choose **Select key Pair** from the dropdown list.

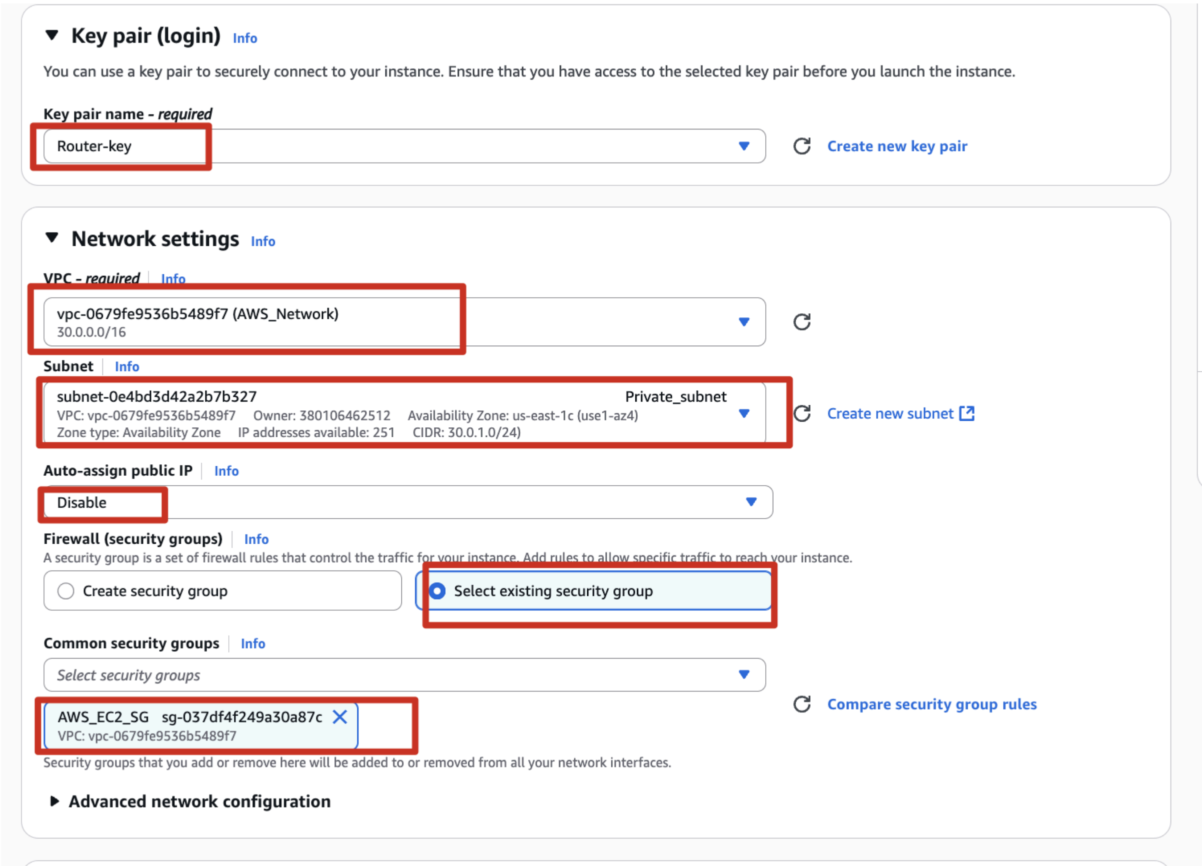
- Key pair name : Enter ***Route-key***

5. Under **Network Settings,** click on **Edit** button.

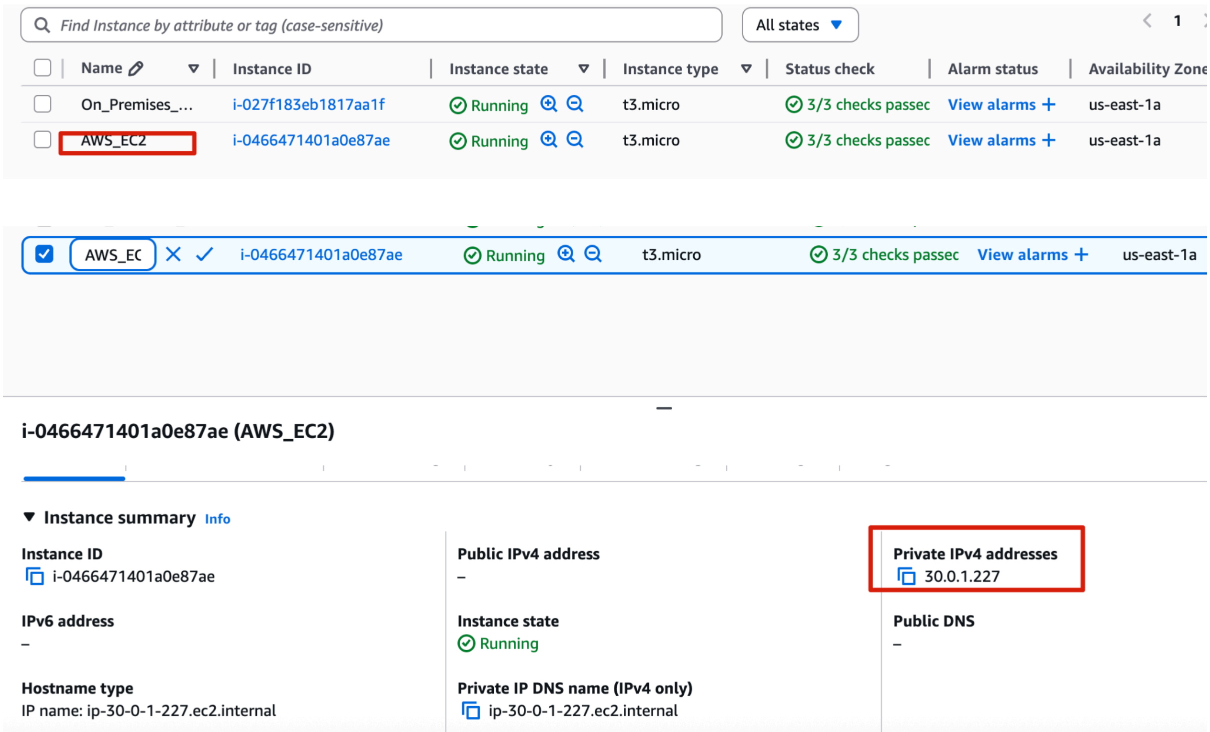
- Network : Select **AWS_Network**
- Subnet : leave as default
- Auto-assign Public IP : Select **Disable**

6. Configure Security Group:

- Firewall(security groups) : Select **existing security group**
- Select ***AWS_EC2_SG***

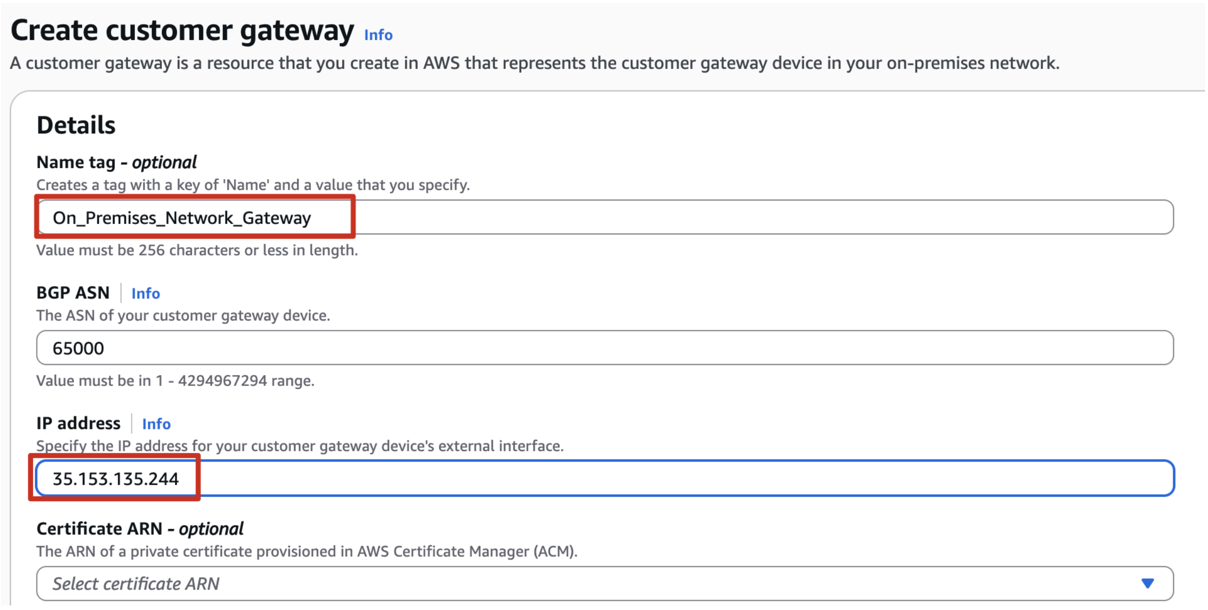


7. Click on **Launch Instance** button.

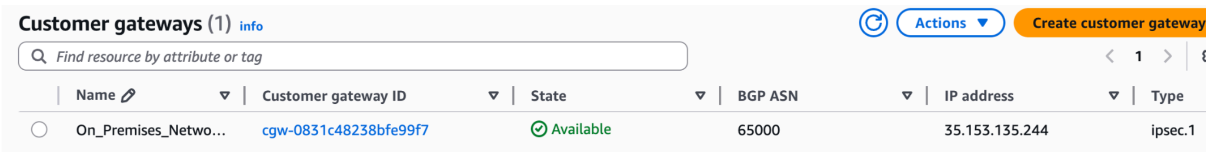


Step 12: Create a Customer Gateway

1. **VPC→Customer Gateway→Virtual Private Network (VPN)**.
2. Click on the **Create Customer Gateway**
   - Name : Enter ***On_Premises_Network_Gateway***
   - IP Address : Enter **On-premises EC2 instance IPv4 Public IP (EC2 act as the Public Router)**
   - Leave everything else as default.



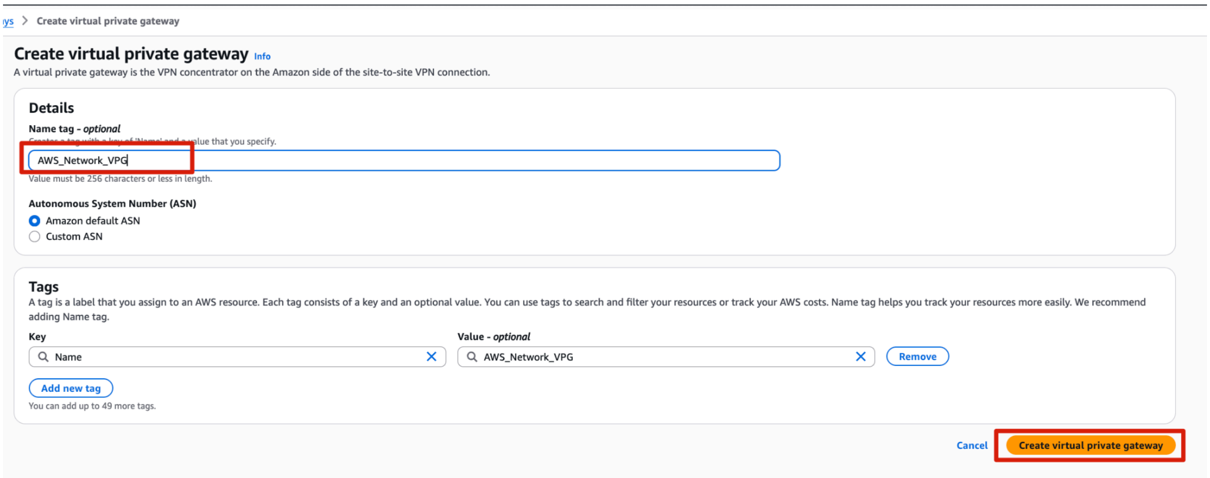
Click on the **Create Customer Gateway**



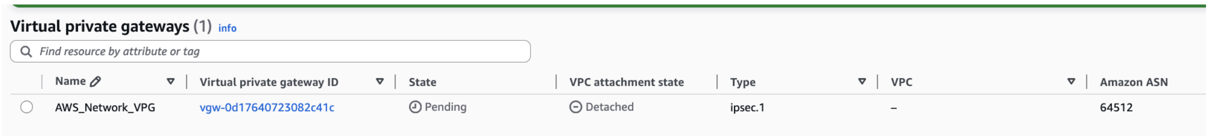
Step 13: Create a Virtual Private Gateway

1. Now scroll down and select **Virtual Private Gateways** under **Virtual Private Network (VPN)**.
2. Click on the **Create Virtual Private Gateway**
   - Name tag : Enter ***AWS_Network_VPG***
   - ASN : Leave as Default

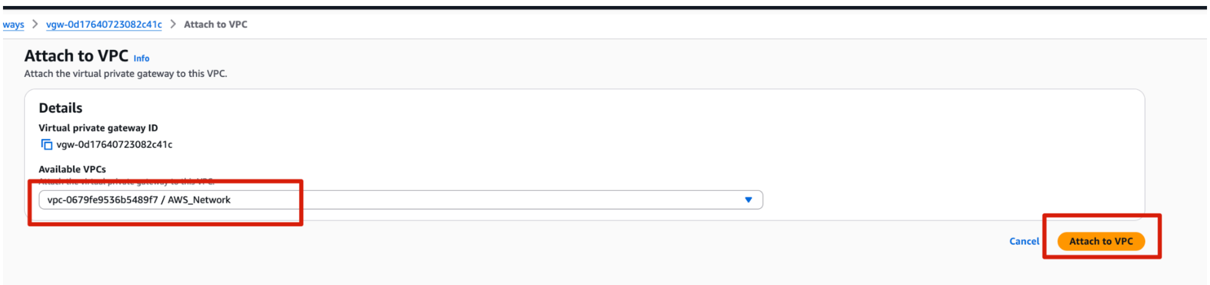
3. Now click on the **Create Virtual Private Gateway**



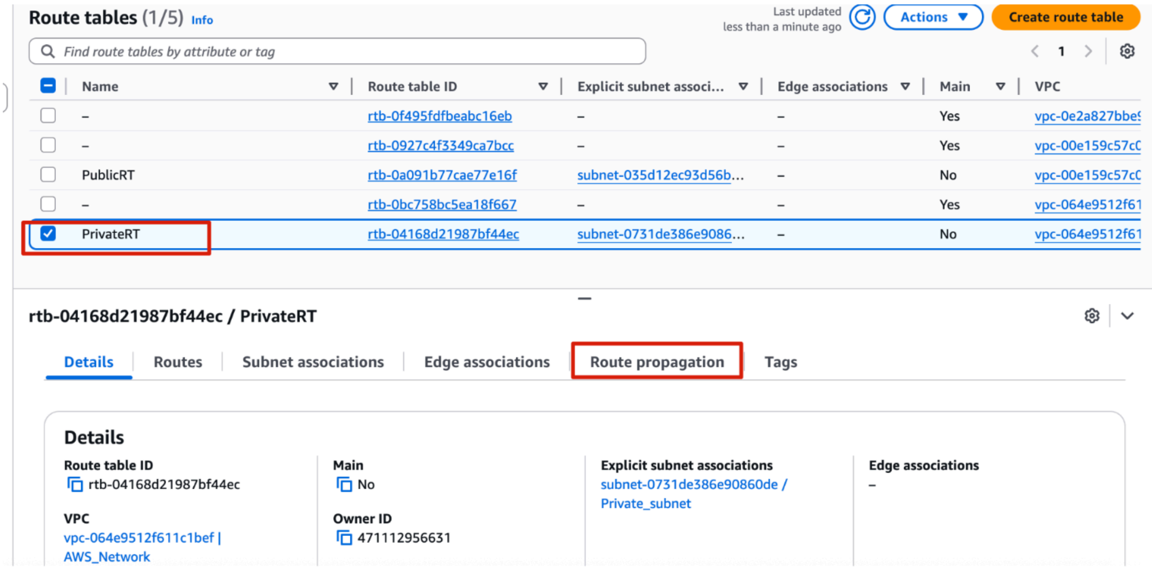
4. Now the state will be **detached** for the Virtual Private gateway that you have created.



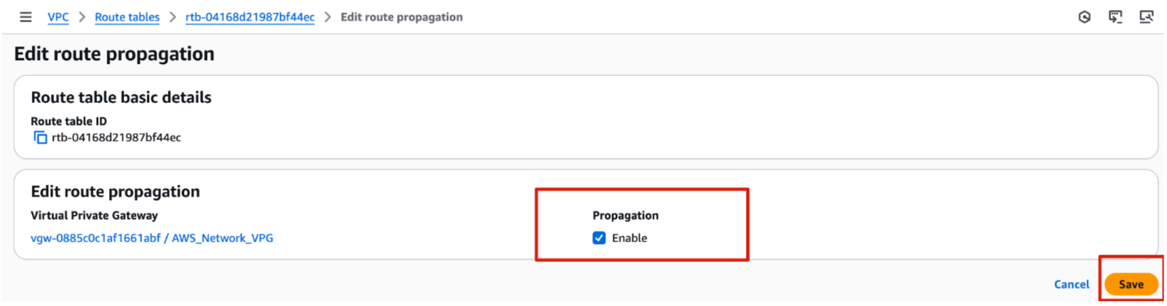
5. You need to attach the VPG to the VPC
6. Select the **Virtual Private gateway** that you just created (**wait till the state changes to Available**), click on the **Actions** button and select **Attach to VPC.**
   - VPC : Select **AWS_Network** from the list.
   - Now click on the **Attach to VPC**



Now select the PrivateRT **RouteTable**, where VPC is mentioned as **AWS_Network**. And goto the **Route propagation** tab below and click on the **Edit route propagation** button.



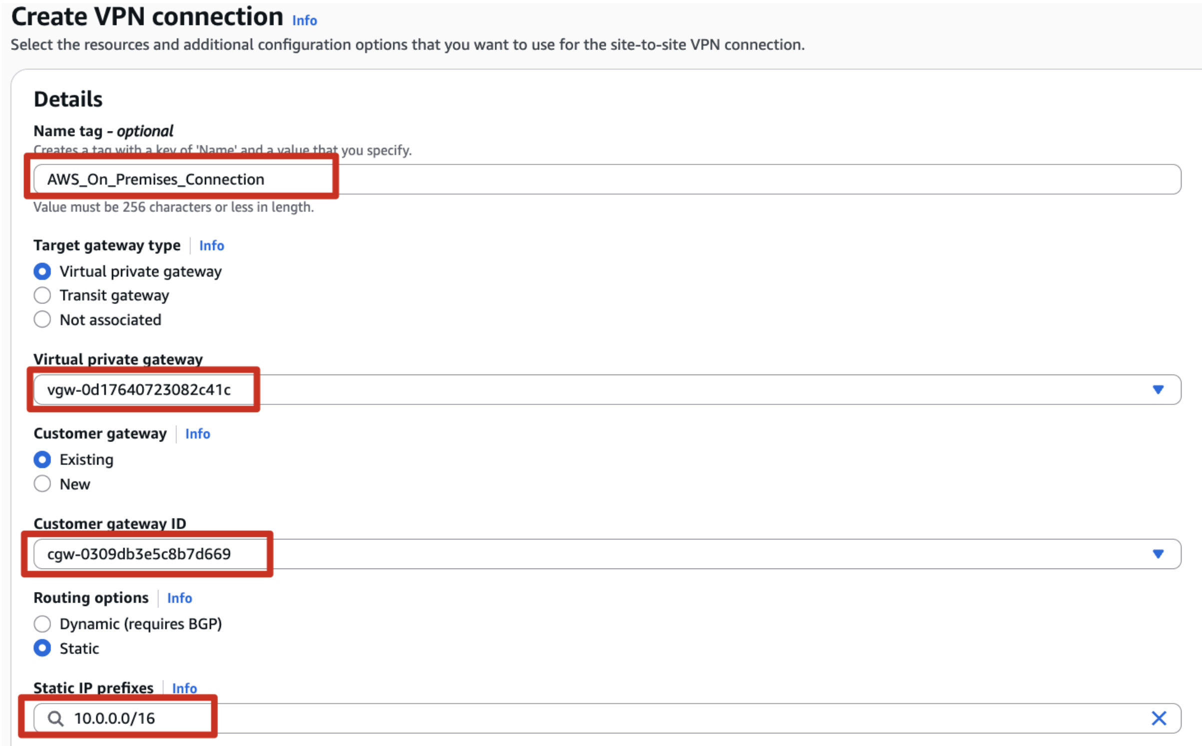
Check the **Enable** checkbox and click on the **Save** button.



Step 14: Create a Site-to-Site VPN connection

1. **VPC→Site-to-Site VPN Connections→Virtual Private Network(VPN)-->Create VPN Connection**

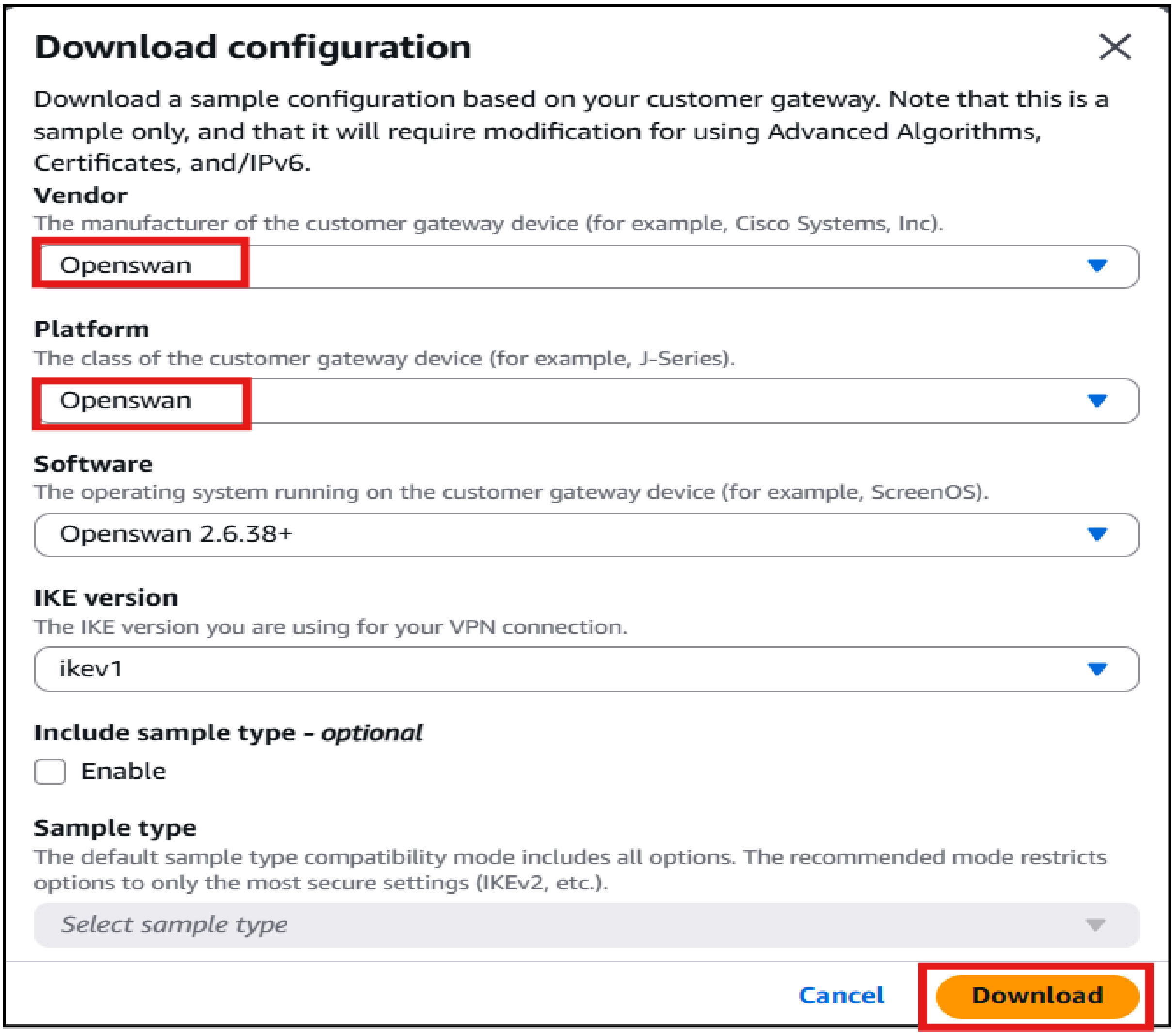
- Name tag : Enter ***AWS_On_Premises_Connection***
- Virtual Private Gateway* : Select **AWS_Network_VPG** from the list.
- Customer Gateway ID* : Select **On_Premises_Network** from the list.
- Routing Options : Select **Static**
- Static IP Prefixes : Enter ***10.0.0.0/16*** in the textbox (Your On-Premises VPC CIDR)



- Tunnel Options :
    - Leave as default.

3. Click on the **Create VPN Connection** .It will take 3 to 4 Minutes to set up the VPN Connection.

4. Select the VPN Connection and on top, click on the **Download the configuration**
    a. Vendor : Select **Openswan**
    b. Platform : Select **Openswan**
    c. Software : Leave default.



5. Click on the **Download** button and the configuration file will be downloaded to your local machine and note down the file name.

Step 15: Configure On-Premises Router

1. You need to SSH into the EC2 instance (**On_Premises_Router**) that you have created in the **On_Premises_Network**. This EC2 will act as the Public Router of On-Premises Network.
2. SSH to EC2 instance

```
ssh -i "Router-key.pem" ec2-user@35.153.135.244
```

Switch to root user :

```
sudo -s
```

3. Install Openswan

```
sudo dnf install libreswan -y
```

4. Next make sure the last line in **/etc/ipsec.conf** is not commented. (NO **#** in the beginning)

```
nano  /etc/ipsec.conf
```

- Scroll to the end and make sure the last line include **/etc/ipsec.d/*.conf** has no hash **(#)** in the beginning.

```
# if it exists, include system wide crypto-policy def
# include /etc/crypto-policies/back-ends/libreswan.co

# It is best to add your IPsec connections as separat
include /etc/ipsec.d/*.conf
```

- Click **[Control] + X** or **[Ctrl] + X** to exit the file.

5. Update **/etc/sysctl.conf** file

```
nano  /etc/sysctl.conf
```

- Add the below 3 lines in end of this file with no hash (#) in the beginning and add each in new lines

```
net.ipv4.ip_forward = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
```

```
# sysctl settings are defined through files in
# /usr/lib/sysctl.d/, /run/sysctl.d/, and /etc/sysctl.d/.
#
# Vendors settings live in /usr/lib/sysctl.d/.
# To override a whole file, create a new file with the same in
# /etc/sysctl.d/ and put new settings there. To override
# only specific settings, add a file with a lexically later
# name in /etc/sysctl.d/ and put new settings there.
#
# For more information, see sysctl.conf(5) and sysctl.d(5).
net.ipv4.ip_forward = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
```

- Click **[Control] + X** or **[Ctrl] + X** to exit the file.
- Save modified buffer (Answering "No" will DISCARD changes.) : Enter **Y**
- File Name to Write: /etc/sysctl.conf : Hit **[Enter]** Key

6. Restart Network Service

```
sudo systemctl restart systemd-networkd
```

7. Next we need to configure IPSec and pre-shared keys in Openswan.
8. For that you have **downloaded** a **configuration file** from the Site-to-site VPN page to your **local system**. Open that file in your text editor.
9. Create or open **/etc/ipsec.d/aws.conf** file

```
vi  /etc/ipsec.d/aws.conf
```

- In the Configuration file, look for **point number 4** under **IPSEC Tunnel #1** and copy the entire code below **point number 4**.

```
--------------------------------------------------------------------------------------------------------------------
IPSEC Tunnel #1
--------------------------------------------------------------------------------------------------------------------

This configuration assumes that you already have a default openswan installation in place on the Amazon Linux operating system (but may also work with other distros as well)

1) Open /etc/sysctl.conf and ensure that its values match the following:
   net.ipv4.ip_forward = 1
   net.ipv4.conf.default.rp_filter = 0
   net.ipv4.conf.default.accept_source_route = 0

2) Apply the changes in step 1 by executing the command 'sysctl -p'

3) Open /etc/ipsec.conf and look for the line below. Ensure that the # in front of the line has been removed, then save and exit the file.
    #include /etc/ipsec.d/*.conf

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration to the end in the file:
 #leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with this value (don't include the brackets). If you have m
0.0.0.0/0 instead.
 #rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you will need to replace <REMOTE NETWORK> with this value (

conn Tunnel1
        authby=secret
        auto=start
        left=%defaultroute
        leftid=35.153.135.244
        right=34.192.156.108
        type=tunnel
        ikelifetime=8h
        keylife=1h
        phase2alg=aes128-sha1;modp1024
        ike=aes128-sha1;modp1024
        auth=esp
        keyingtries=%forever
        keyexchange=ike
        leftsubnet=<LOCAL NETWORK>
        rightsubnet=<REMOTE NETWORK>
        dpddelay=10
        dpdtimeout=30
        dpdaction=restart_by_peer
5) Create a new file at /etc/ipsec.d/aws.secrets if it doesn't already exist, and append this line to the file (be mindful of the spacing!):
35.153.135.244 34.192.156.108: PSK "5.a8HhH8HreytbEvd9D4XZrLEZSLuPPq"
```

- Press " **i** " button in your keyboard to edit the file you have created.
- Paste the code in the file we opened.
- **Remove** the line **auth=esp** from the file (else the connection won't work)

```
conn Tunnel1
        authby=secret
        auto=start
        left=%defaultroute
        leftid=35.153.135.244
        right=34.192.156.108
        type=tunnel
        ikelifetime=8h
        keylife=1h
        phase2alg=aes128-sha1;modp2048
        ike=aes128-sha1;modp2048
        keyingtries=%forever
        keyexchange=ike
        leftsubnet=10.0.0.0/16
        rightsubnet=30.0.0.0/16
        dpddelay=10
        dpdtimeout=30
        dpdaction=restart_by_peer
```

- Update
    - **phase2alg=aes128-sha1;modp1024**
    - **ike=aes128-sha1;modp1024**
- To
    - **phase2alg=aes128-sha1;modp2048**
    - **ike=aes128-sha1;modp2048**
- Update **leftsubnet** and **rightsubnet**
    - leftsubnet= Enter ***10.0.0.0/16*** (On-premises VPC CIDR)
    - rightsubnet= Enter ***30.0.0.0/16*** (AWS VPC CIDR)

```
conn Tunnel1
        authby=secret
        auto=start
        left=%defaultroute
        leftid=35.153.135.244
        right=34.192.156.108
        type=tunnel
        ikelifetime=8h
        keylife=1h
        phase2alg=aes128-sha1;modp2048
        ike=aes128-sha1;modp2048
        keyingtries=%forever
        keyexchange=ike
        leftsubnet=10.0.0.0/16
        rightsubnet=30.0.0.0/16
        dpddelay=10
        dpdtimeout=30
        dpdaction=restart_by_peer
```

- Click **[Esc]** button in the keyboard to exit the editing mode.
- Now type **:wq** and hit **[Enter]** Key to save the file.

10. Create or open **/etc/ipsec.d/aws.secrets** file

```
vi  /etc/ipsec.d/aws.secrets
```

- In the Configuration file, look for **point number 5** under **IPSEC Tunnel #1** and copy the entire code below.

```
1) Open /etc/sysctl.conf and ensure that its values match the following:
   net.ipv4.ip_forward = 1
   net.ipv4.conf.default.rp_filter = 0
   net.ipv4.conf.default.accept_source_route = 0

2) Apply the changes in step 1 by executing the command 'sysctl -p'

3) Open /etc/ipsec.conf and look for the line below. Ensure that the # in front of the line has been removed, then save and
    #include /etc/ipsec.d/*.conf

4) Create a new file at /etc/ipsec.d/aws.conf if doesn't already exist, and then open it. Append the following configuration
 #leftsubnet= is the local network behind your openswan server, and you will need to replace the <LOCAL NETWORK> below with
0.0.0.0/0 instead.
 #rightsubnet= is the remote network on the other side of your VPN tunnel that you wish to have connectivity with, and you w

conn Tunnel1
        authby=secret
        auto=start
        left=%defaultroute
        leftid=35.153.135.244
        right=34.192.156.108
        type=tunnel
        ikelifetime=8h
        keylife=1h
        phase2alg=aes128-sha1;modp1024
        ike=aes128-sha1;modp1024
        auth=esp
        keyingtries=%forever
        keyexchange=ike
        leftsubnet=<LOCAL NETWORK>
        rightsubnet=<REMOTE NETWORK>
        dpddelay=10
        dpdtimeout=30
        dpdaction=restart_by_peer

5) Create a new file at /etc/ipsec.d/aws.secrets if it doesn't already exist, and append this line to the file (be mindful o
35.153.135.244 34.192.156.108: PSK "5.a8HhH8HreytbEvd9D4XZrLEZSLuPPq"
```

- Press " **i** " button in your keyboard to edit the file you have created.
- Paste the secret key in the file you opened.

```
vi  /etc/ipsec.d/aws.secrets
```

```
35.153.135.244 34.192.156.108: PSK "5.a8HhH8HreytbEvd9D4XZrLEZSLuPPq"
```

- Click **[Esc]** button in the keyboard to exit the editing mode.
- Now type **:wq** and hit **[Enter]** Key to save the file.

11. In case if your EC2 Session gets timed out. please follow the steps to SSH into EC2 Instance again.
    - Switch to root user :

```
sudo -s
```

12. Start IPSec service

```
systemctl start ipsec
```

13. Check the status of IPSec

```
systemctl status ipsec
```

```
[root@ip-10-0-1-12 ec2-user]# sudo -s
[root@ip-10-0-1-12 ec2-user]# systemctl start ipsec
[root@ip-10-0-1-12 ec2-user]# systemctl status ipsec
● ipsec.service - Internet Key Exchange (IKE) Protocol Daemon for IPsec
   Loaded: loaded (/usr/lib/systemd/system/ipsec.service; disabled; vendor preset: disabled)
   Active: active (running) since Tue 2025-06-03 06:37:30 UTC; 4s ago
     Docs: man:ipsec(8)
           man:pluto(8)
           man:ipsec.conf(5)
  Process: 4491 ExecStartPre=/usr/sbin/ipsec --checknflog (code=exited, status=0/SUCCESS)
  Process: 4484 ExecStartPre=/usr/sbin/ipsec --checknss (code=exited, status=0/SUCCESS)
  Process: 3968 ExecStartPre=/usr/libexec/ipsec/_stackmanager start (code=exited, status=0/SUCCESS)
  Process: 3967 ExecStartPre=/usr/libexec/ipsec/addconn --config /etc/ipsec.conf --checkconfig (code=exited, status=0/SUCCESS)
 Main PID: 4509 (pluto)
   Status: "Startup completed."
   CGroup: /system.slice/ipsec.service
           └─4509 /usr/libexec/ipsec/pluto --leak-detective --config /etc/ipsec.conf --nofork
```

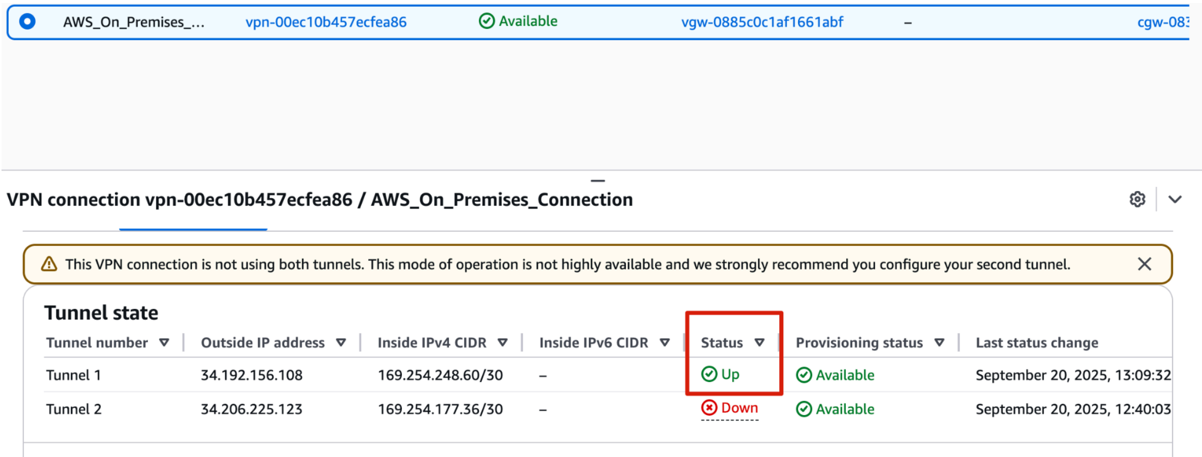
Step 16: Test the connectivity between two Networks

1. Ping **OnPrem EC2 to AWS Private EC2**
   - **ping** <Private IPv4 Address of On_Premises_Router>
   - Example : **ping 30.0.1.227**

```
[root@ip-10-0-1-102 ec2-user]# ping 30.0.1.227
PING 30.0.1.227 (30.0.1.227) 56(84) bytes of data.
64 bytes from 30.0.1.227: icmp_seq=1 ttl=127 time=1.81 ms
64 bytes from 30.0.1.227: icmp_seq=2 ttl=127 time=1.77 ms
64 bytes from 30.0.1.227: icmp_seq=3 ttl=127 time=1.71 ms
64 bytes from 30.0.1.227: icmp_seq=4 ttl=127 time=1.94 ms
^C
```

2. Now scroll down and select **Site-to-Site VPN Connection** under **Virtual Private Network(VPC)**.
3. Check the **Tunnel Details** and you will be able to see that **Tunnel 1** is **UP**. Tunnel 2 is Down because in Openswan only one tunnel can be configured and we only used

Tunnel 1.



Step 17: Delete AWS Resources

1. Terminate both EC2 instances (On_Premises_Router and AWS_EC2).
2. Delete the Security Groups created for the lab (On_Premises_Router_SG and AWS_EC2_SG).
3. Detach and delete the Internet Gateway from the On_Premises_Network VPC.
4. Delete the subnets (Public_subnet and Private_subnet).
5. Delete the Route Tables (PublicRT and PrivateRT).
6. Delete the Customer Gateway and the Virtual Private Gateway.
7. Delete the Site-to-Site VPN Connection.
8. Finally, delete both VPCs (On_Premises_Network and AWS_Network).